## Mini_PROJECT-6 — Enterprise Multi-Agent AI Dashboard

## Project Name: AgentOps AI — Enterprise Multi-Agent AI Dashboard

**Problem Statement**

Enterprises have multiple departments like HR, IT, Finance, Security, and Operations. Each department has different workflows, but users usually need one unified AI interface.

The goal is to build a **multi-agent enterprise dashboard** where one parent orchestrator agent understands the user request and routes it to the correct specialized AI agent.

---

### 1. Project Features

| Feature | Description |
|---|---|
| Multi-Agent Chat | One interface for HR, IT, Finance, Governance queries |
| Workflow Orchestration | Parent agent routes task to child agents |
| AI Analytics Dashboard | Track requests by department |
| Token Monitoring | Track prompt/output token usage |
| Governance Logs | Store every AI decision |
| Notification Agent | Send alerts for high-priority tasks |
| Role-Based Design | Enterprise-ready security pattern |

---

### 2. AI Agents

| Agent | Purpose |
|---|---|
| Orchestrator Agent | Classifies request and routes to correct agent |
| HR Agent | HR policy, leave, onboarding, payroll queries |
| IT Agent | Incident triage, troubleshooting, ticket routing |
| Finance Agent | Expense approval, budget validation, risk analysis |
| Governance Agent | Security, compliance, policy monitoring |
| Notification Agent | Sends Teams/Email alerts |

---

### 3. Architecture

User
↓
React Dashboard
↓
Node.js Backend API
↓
Orchestrator Agent
↓
HR / IT / Finance / Governance Agent
↓
MongoDB Logs + Analytics
↓
Power Automate / Teams / Outlook Notifications

---

### 4. Tech Stack

| Layer | Technology |
|---|---|
| Frontend | React |
| Backend | Node.js + Express |
| Database | MongoDB |
| AI | Azure OpenAI |
| Analytics | MongoDB aggregation |
| Notification | Power Automate Webhook |
| Security | Entra ID optional |

---

### 5. Folder Structure

```
agentops-ai/
│
├── client/
│   └── src/
│       ├── App.js
│       ├── pages/
│       │   ├── ChatPage.js
│       │   └── Dashboard.js
```

```
│       └── services/
│           └── api.js
│
└── server/
    ├── server.js
    ├── .env
    ├── routes/
    │   ├── agentRoutes.js
    │   └── analyticsRoutes.js
    ├── controllers/
    │   ├── agentController.js
    │   └── analyticsController.js
    ├── models/
    │   └── AgentLog.js
    └── agents/
        ├── azureOpenAI.js
        ├── orchestratorAgent.js
        ├── hrAgent.js
        ├── itAgent.js
        ├── financeAgent.js
        ├── governanceAgent.js
        └── notificationAgent.js
```

---

**6. Backend Setup**

```
mkdir agentops-ai
cd agentops-ai

mkdir server
cd server

npm init -y
npm install express cors dotenv axios mongoose
```

---

**server/.env**

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/agentops_ai

AZURE_OPENAI_ENDPOINT=https://YOUR-RESOURCE.openai.azure.com
AZURE_OPENAI_KEY=YOUR_KEY
AZURE_OPENAI_DEPLOYMENT=YOUR_DEPLOYMENT_NAME
```

```
AZURE_OPENAI_API_VERSION=2024-02-15-preview

POWER_AUTOMATE_WEBHOOK_URL=YOUR_POWER_AUTOMATE_WEBHOOK_
URL
```

---

**server/server.js**

```
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
require("dotenv").config();

const agentRoutes = require("./routes/agentRoutes");
const analyticsRoutes = require("./routes/analyticsRoutes");

const app = express();

app.use(cors());
app.use(express.json());

app.use("/api/agents", agentRoutes);
app.use("/api/analytics", analyticsRoutes);

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB connected"))
  .catch((error) => console.log("MongoDB error:", error.message));

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`AgentOps AI server running on port ${PORT}`);
});
```

---

**7. MongoDB Log Model**

**server/models/AgentLog.js**

```
const mongoose = require("mongoose");

const AgentLogSchema = new mongoose.Schema(
  {
    userRequest: String,
```

```
    selectedAgent: String,
    response: String,
    riskLevel: String,
    inputTokens: Number,
    outputTokens: Number,
    totalTokens: Number,
    status: {
      type: String,
      default: "Completed"
    }
  },
  {
    timestamps: true
  }
);

module.exports = mongoose.model("AgentLog", AgentLogSchema);
```

---

**8. Azure OpenAI Common Service**

**server/agents/azureOpenAI.js**

```
const axios = require("axios");

async function callAzureOpenAI(systemPrompt, userPrompt) {
  const url =
    `${process.env.AZURE_OPENAI_ENDPOINT}/openai/deployments/` +
    `${process.env.AZURE_OPENAI_DEPLOYMENT}/chat/completions` +
    `?api-version=${process.env.AZURE_OPENAI_API_VERSION}`;

  const response = await axios.post(
    url,
    {
      messages: [
        {
          role: "system",
          content: systemPrompt
        },
        {
          role: "user",
          content: userPrompt
        }
      ],
```

```
    temperature: 0.3
   },
   {
    headers: {
     "api-key": process.env.AZURE_OPENAI_KEY,
     "Content-Type": "application/json"
    }
   }
  );

  return {
   content: response.data.choices[0].message.content,
   usage: response.data.usage || {
    prompt_tokens: 0,
    completion_tokens: 0,
    total_tokens: 0
   }
  };
}

module.exports = {
  callAzureOpenAI
};
```

---

**9. Orchestrator Agent**

**server/agents/orchestratorAgent.js**

```
const { callAzureOpenAI } = require("./azureOpenAI");

async function classifyAgent(userRequest) {
  const systemPrompt = `
You are an enterprise AI orchestrator.

Classify the user request into exactly one category:

HR
IT
Finance
Governance
General

Return only one word.
```

```
`;

  const result = await callAzureOpenAI(systemPrompt, userRequest);

  return result.content.trim();
}

module.exports = {
  classifyAgent
};
```

---

**10. HR Agent**

**server/agents/hrAgent.js**

```
const { callAzureOpenAI } = require("./azureOpenAI");

async function hrAgent(userRequest) {
  const systemPrompt = `
You are an enterprise HR AI agent.

You help with:
- Leave policy
- Payroll queries
- Employee onboarding
- Benefits
- HR compliance

Give clear enterprise-style answers.
`;

  return await callAzureOpenAI(systemPrompt, userRequest);
}

module.exports = {
  hrAgent
};
```

---

**11. IT Agent**

**server/agents/itAgent.js**

```
const { callAzureOpenAI } = require("./azureOpenAI");

async function itAgent(userRequest) {
  const systemPrompt = `
You are an enterprise IT support AI agent.

You help with:
- Incident triage
- Laptop issues
- VPN problems
- Outlook/Teams issues
- Password reset guidance
- Ticket routing

Return troubleshooting steps and severity.
`;

  return await callAzureOpenAI(systemPrompt, userRequest);
}

module.exports = {
  itAgent
};
```

---

**12. Finance Agent**

**server/agents/financeAgent.js**

```
const { callAzureOpenAI } = require("./azureOpenAI");

async function financeAgent(userRequest) {
  const systemPrompt = `
You are an enterprise Finance AI agent.

You help with:
- Expense approvals
- Budget validation
- Compliance risk
- Fraud indicators
- Approval recommendations

Give finance-safe, policy-aware recommendations.
`;
```

```
  return await callAzureOpenAI(systemPrompt, userRequest);
}

module.exports = {
  financeAgent
};
```

---

**13. Governance Agent**

**server/agents/governanceAgent.js**

```
const { callAzureOpenAI } = require("./azureOpenAI");

async function governanceAgent(userRequest) {
  const systemPrompt = `
You are an enterprise AI governance and security agent.

You help with:
- RBAC
- Responsible AI
- Compliance
- Data privacy
- Security monitoring
- Prompt injection risk
- Audit controls

Give security-first recommendations.
`;

  return await callAzureOpenAI(systemPrompt, userRequest);
}

module.exports = {
  governanceAgent
};
```

---

**14. Notification Agent**

**server/agents/notificationAgent.js**

```
const axios = require("axios");
```

```
async function notificationAgent(payload) {
  if (!process.env.POWER_AUTOMATE_WEBHOOK_URL) {
    return {
      sent: false,
      message: "Power Automate webhook not configured"
    };
  }

  await axios.post(process.env.POWER_AUTOMATE_WEBHOOK_URL, payload);

  return {
    sent: true,
    message: "Notification sent successfully"
  };
}

module.exports = {
  notificationAgent
};
```

---

**15. Main Agent Controller**

**server/controllers/agentController.js**

```
const AgentLog = require("../models/AgentLog");

const { classifyAgent } = require("../agents/orchestratorAgent");
const { hrAgent } = require("../agents/hrAgent");
const { itAgent } = require("../agents/itAgent");
const { financeAgent } = require("../agents/financeAgent");
const { governanceAgent } = require("../agents/governanceAgent");
const { notificationAgent } = require("../agents/notificationAgent");

exports.processRequest = async (req, res) => {
  try {
    const { userRequest } = req.body;

    const selectedAgent = await classifyAgent(userRequest);

    let result;

    if (selectedAgent === "HR") {
      result = await hrAgent(userRequest);
```

```
} else if (selectedAgent === "IT") {
  result = await itAgent(userRequest);
} else if (selectedAgent === "Finance") {
  result = await financeAgent(userRequest);
} else if (selectedAgent === "Governance") {
  result = await governanceAgent(userRequest);
} else {
  result = {
    content: "I could not map this request to HR, IT, Finance, or Governance.",
    usage: {
      prompt_tokens: 0,
      completion_tokens: 0,
      total_tokens: 0
    }
  };
}

let riskLevel = "Low";

if (
  userRequest.toLowerCase().includes("security") ||
  userRequest.toLowerCase().includes("breach") ||
  userRequest.toLowerCase().includes("urgent")
) {
  riskLevel = "High";
}

const log = await AgentLog.create({
  userRequest,
  selectedAgent,
  response: result.content,
  riskLevel,
  inputTokens: result.usage.prompt_tokens,
  outputTokens: result.usage.completion_tokens,
  totalTokens: result.usage.total_tokens
});

if (riskLevel === "High") {
  await notificationAgent({
    title: "High Risk AI Request",
    agent: selectedAgent,
    userRequest,
    response: result.content
```

```
    });
   }

   res.json({
    selectedAgent,
    response: result.content,
    riskLevel,
    tokens: result.usage,
    logId: log._id
   });
  } catch (error) {
   res.status(500).json({
    message: "Multi-agent processing failed",
    error: error.message
   });
  }
};
```

---

**16. Analytics Controller**

**server/controllers/analyticsController.js**

```
const AgentLog = require("../models/AgentLog");

exports.getAnalytics = async (req, res) => {
  try {
   const totalRequests = await AgentLog.countDocuments();

   const requestsByAgent = await AgentLog.aggregate([
    {
     $group: {
      _id: "$selectedAgent",
      count: { $sum: 1 }
     }
    }
   ]);

   const tokenUsage = await AgentLog.aggregate([
    {
     $group: {
      _id: null,
      totalInputTokens: { $sum: "$inputTokens" },
      totalOutputTokens: { $sum: "$outputTokens" },
```

```
        totalTokens: { $sum: "$totalTokens" }
      }
    }
  ]);

  const highRiskRequests = await AgentLog.countDocuments({
    riskLevel: "High"
  });

  res.json({
    totalRequests,
    requestsByAgent,
    tokenUsage: tokenUsage[0] || {
      totalInputTokens: 0,
      totalOutputTokens: 0,
      totalTokens: 0
    },
    highRiskRequests
  });
 } catch (error) {
  res.status(500).json({
    message: "Analytics fetch failed",
    error: error.message
  });
 }
};

exports.getLogs = async (req, res) => {
 const logs = await AgentLog.find().sort({ createdAt: -1 }).limit(20);
 res.json(logs);
};
```

---

**17. Routes**

**server/routes/agentRoutes.js**

```
const express = require("express");
const router = express.Router();

const {
 processRequest
} = require("../controllers/agentController");
```

```
router.post("/process", processRequest);

module.exports = router;
```

**server/routes/analyticsRoutes.js**

```
const express = require("express");
const router = express.Router();

const {
  getAnalytics,
  getLogs
} = require("../controllers/analyticsController");

router.get("/", getAnalytics);
router.get("/logs", getLogs);

module.exports = router;
```

---

**18. Frontend Setup**

```
cd ..
npx create-react-app client
cd client
npm install axios
```

---

**client/src/services/api.js**

```
import axios from "axios";

const API_BASE_URL = "http://localhost:5000/api";

export async function processAgentRequest(userRequest) {
  const response = await axios.post(`${API_BASE_URL}/agents/process`, {
    userRequest
  });

  return response.data;
}

export async function getAnalytics() {
  const response = await axios.get(`${API_BASE_URL}/analytics`);
  return response.data;
```

```
}

export async function getLogs() {
  const response = await axios.get(`${API_BASE_URL}/analytics/logs`);
  return response.data;
}
```

---

**19. React App**

**client/src/App.js**

```
import React from "react";
import ChatPage from "./pages/ChatPage";
import Dashboard from "./pages/Dashboard";

function App() {
  return (
    <div style={{ padding: "30px", fontFamily: "Arial" }}>
      <h1>AgentOps AI</h1>
      <p>Enterprise Multi-Agent AI Dashboard</p>

      <ChatPage />
      <hr />
      <Dashboard />
    </div>
  );
}

export default App;
```

---

**client/src/pages/ChatPage.js**

```
import React, { useState } from "react";
import { processAgentRequest } from "../services/api";

function ChatPage() {
  const [userRequest, setUserRequest] = useState("");
  const [result, setResult] = useState(null);

  async function handleSubmit(e) {
    e.preventDefault();

    const data = await processAgentRequest(userRequest);
```

```
    setResult(data);
  }

  return (
    <div>
      <h2>Multi-Agent Chat</h2>

      <form onSubmit={handleSubmit}>
        <textarea
          rows="5"
          style={{ width: "100%" }}
          placeholder="Example: My laptop is slow and Teams is crashing..."
          value={userRequest}
          onChange={(e) => setUserRequest(e.target.value)}
        />

        <br /><br />

        <button type="submit">
          Send to Orchestrator Agent
        </button>
      </form>

      {result && (
        <div style={{ marginTop: "20px", padding: "15px", border: "1px solid #ccc" }}>
          <h3>Agent Response</h3>
          <p><b>Selected Agent:</b> {result.selectedAgent}</p>
          <p><b>Risk Level:</b> {result.riskLevel}</p>
          <p><b>Response:</b></p>
          <pre style={{ whiteSpace: "pre-wrap" }}>{result.response}</pre>

          <h4>Token Usage</h4>
          <p><b>Input Tokens:</b> {result.tokens.prompt_tokens}</p>
          <p><b>Output Tokens:</b> {result.tokens.completion_tokens}</p>
          <p><b>Total Tokens:</b> {result.tokens.total_tokens}</p>
        </div>
      )}
    </div>
  );
}

export default ChatPage;
```

---

**client/src/pages/Dashboard.js**

```
import React, { useEffect, useState } from "react";
import { getAnalytics, getLogs } from "../services/api";

function Dashboard() {
  const [analytics, setAnalytics] = useState(null);
  const [logs, setLogs] = useState([]);

  async function loadDashboard() {
    const analyticsData = await getAnalytics();
    const logsData = await getLogs();

    setAnalytics(analyticsData);
    setLogs(logsData);
  }

  useEffect(() => {
    loadDashboard();
  }, []);

  return (
    <div>
      <h2>AI Analytics Dashboard</h2>

      <button onClick={loadDashboard}>Refresh Dashboard</button>

      {analytics && (
        <div style={{ display: "flex", gap: "20px", marginTop: "20px" }}>
          <div style={{ padding: "15px", border: "1px solid #ccc" }}>
            <h3>Total Requests</h3>
            <p>{analytics.totalRequests}</p>
          </div>

          <div style={{ padding: "15px", border: "1px solid #ccc" }}>
            <h3>Total Tokens</h3>
            <p>{analytics.tokenUsage.totalTokens}</p>
          </div>

          <div style={{ padding: "15px", border: "1px solid #ccc" }}>
            <h3>High Risk Requests</h3>
            <p>{analytics.highRiskRequests}</p>
          </div>
        </div>
```

```
      )}

      <h3>Requests by Agent</h3>

      {analytics && (
        <ul>
          {analytics.requestsByAgent.map((item) => (
            <li key={item._id}>
              {item._id}: {item.count}
            </li>
          ))}
        </ul>
      )}

      <h3>Recent Governance Logs</h3>

      <table border="1" cellPadding="10" style={{ width: "100%" }}>
        <thead>
          <tr>
            <th>Agent</th>
            <th>Risk</th>
            <th>Request</th>
            <th>Tokens</th>
          </tr>
        </thead>

        <tbody>
          {logs.map((log) => (
            <tr key={log._id}>
              <td>{log.selectedAgent}</td>
              <td>{log.riskLevel}</td>
              <td>{log.userRequest}</td>
              <td>{log.totalTokens}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default Dashboard;
```

---

**20. Run Project**

**Terminal 1 — Start MongoDB**

mongod

**Terminal 2 — Start Backend**

cd agentops-ai/server
node server.js

**Terminal 3 — Start Frontend**

cd agentops-ai/client
npm start

---

**21. Sample Test Requests**

**HR Request**

What is the leave policy for new employees?

Expected routing:

HR Agent

**IT Request**

My VPN is not connecting and Teams keeps crashing.

Expected routing:

IT Agent

**Finance Request**

Please validate this travel expense of 95000 for client visit.

Expected routing:

Finance Agent

**Governance Request**

How do we prevent prompt injection and secure AI agents?

Expected routing:

Governance Agent

---

**22. Power Automate Notification Flow**

**Flow Trigger**

Use:

When an HTTP request is received

**JSON Payload**

```
{
  "title": "High Risk AI Request",
  "agent": "Governance",
  "userRequest": "Security breach reported",
  "response": "Escalation required"
}
```

**Actions**

- Post message in Teams channel
- Send Outlook email to admin
- Store notification in SharePoint list

---

**23. Enterprise Enhancements**

Add these for advanced AB-100 capstone:

- Entra ID login
- RBAC for HR/IT/Finance/Admin users
- Azure AI Search for RAG
- SharePoint document grounding
- Dataverse integration
- Power Automate approvals
- ServiceNow ticket creation
- Token cost calculation
- Agent performance chart
- Prompt injection detector
- Audit export report
- Teams bot integration

---

**Final Outcome**

This project demonstrates:

- Multi-agent orchestration
- Enterprise AI dashboard
- Token monitoring
- Governance logging
- AI workflow routing
- Department-specific AI agents
- Power Automate integration
- AB-100 enterprise solution architecture